Hyperparameter Optimisation with Optuna

Software Quality Seminar

Paul-Arthur MESLIN

March 20, 2026

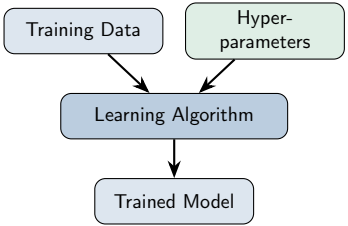
Hyperparameters are the knobs

Parameters

- Learned from data
- Weights, biases, coefficients

Hyperparameters

- Chosen by us
- Learning rate, tree depth, bin width



Any hand-tuned knob creates a search problem.

Why this matters for software quality

Ad-Hoc

- Hard to reproduce
- Choices stay undocumented
- Teams rediscover the same settings

Systematic

- Explicit search space
- Logged trials and outcomes
- Reusable evidence

HPO turns tuning from personal intuition into a reproducible engineering workflow.

Every HPO problem has two parts

Search Space

Continuous, integer, categorical, conditional

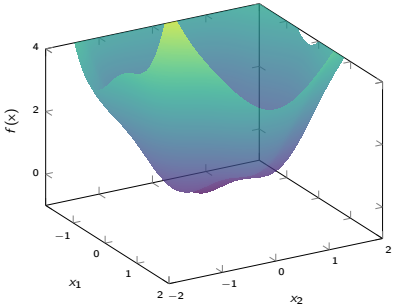
Objective Function

Score each configuration and optimize it

$$x^* = \arg\min_x f(x) \quad \text{or} \quad x^* = \arg\max_x f(x)$$

- x: candidate setting
- $f(x)$: score to minimize or maximize

Same idea for models, simulations, and pipelines.

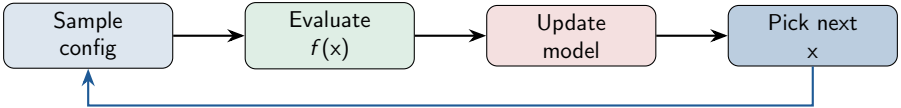


Search landscape with local and global minima.

Bayesian optimisation learns from history

Core Idea

Next trials depend on previous results.



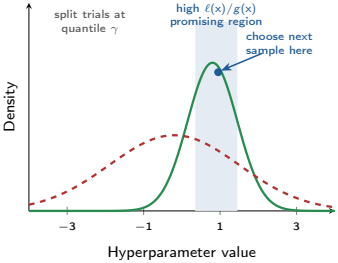
How Optuna sampler works

Optuna does not model $f(x)$ directly.

It fits two densities over previously tried configurations:

- $\ell(x) = p(x \mid y \leq y_\gamma)$ good trials
- $g(x) = p(x \mid y > y_\gamma)$ other trials

where y_γ is the objective threshold at quantile γ.



Acquisition rule:

$$x_{\text{next}} = \arg\max_x \frac{\ell(x)}{g(x)}$$

Interpretation: explore regions that are enriched for good trials and depleted of bad ones.

Optuna: lightweight by design

- Python-native and framework-agnostic
- Define-by-run search spaces
- Pruning, storage, and plots built in

```
pip install optuna
```

Define-by-run

```
def objective(trial):
  n_layers = trial.suggest_int("n_layers", 1, 5)
  dropout = 0.0
  if n_layers > 2:
    dropout = trial.suggest_float("dropout", 0.1,
        0.5)
  score = train_and_evaluate(
    n_layers=n_layers, dropout=dropout)
  return score
```

Putting it together

```python
import optuna

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=50)

print(study.best_params)
print(study.best_value)
```

- `direction` — minimize or maximize
- `n_trials` — your compute budget
- All trials are logged automatically

From theory to workflow

Manual Tuning

- Try a few values by intuition
- Results live in notebooks or memory
- Hard to compare runs fairly
- Best setting is easy to lose

From theory to workflow

Manual Tuning

- Try a few values by intuition
- Results live in notebooks or memory
- Hard to compare runs fairly
- Best setting is easy to lose

Optuna Workflow

- Define search space in code
- Evaluate with one objective function
- Log every trial automatically
- Analyse, resume, prune, and share

It is a reproducible optimisation process.

Demo notebook

1. **Search behaviour:** how trials move through the space
2. **Engineering support:** logging, visualisation, pruning, storage
3. **Trade-offs:** accuracy, time, and multiple objectives

Takeaway

Hyperparameter optimisation is not only about finding a better setting. It is about turning trial-and-error into a transparent, reproducible, and inspectable workflow.

- **Search space** defines what can vary
- **Objective** defines what matters
- **TPE** uses past trials to choose better next trials
- **Optuna** packages this into a practical engineering tool